

Uncertainty-Aware Machine Learning for Solvent Recovery Flowsheet Evaluation

A detailed report with suggestions for future research.

Max Jappert

E-Mail – Website – Code

Contents

1	Introduction	1
1.1	Framing the problem	1
1.2	Proposed solution	1
1.3	Structure of the report	1
1.4	Contributions	2
2	Background	2
2.1	Machine Learning	2
2.2	Deep Learning	3
2.2.1	Hyperparameters	4
2.2.2	Surrogate Models	4
2.2.3	Active Learning	4
2.2.4	Uncertainty Quantification	5
2.3	Model Interpretability	6
3	Methodology	7
3.1	Data	7
3.1.1	Generation	7
3.1.2	Selection	7
3.1.3	Preprocessing	8
3.2	Model Architecture	8
3.3	Active Learning	9
3.4	Metrics	9
3.5	Calibration	9
3.6	Hyperparameter Tuning	10
3.7	Ensemble Training	11
3.8	Visualiser	13
4	Results	13
4.1	Active Learning	13
4.2	Test Losses	14
4.3	Qualitative Analysis	14
5	Discussion	15
5.1	Heteroscedastic deep learning accurately predicts solvent separation metrics	15
5.2	Drawbacks	15
5.3	The case for interpretable models	15
6	Conclusion	16
6.1	Summary	16
6.2	Future work	16
A	Computational Specifications	16
B	Report to Code Mapping	17

Abstract

Superstructure optimisation identifies cost-optimal solvent recovery pathways, but the provided solutions are point estimates. They convey neither the degree of confidence nor the trade-off in terms of other factors between different flowsheets. This report describes and evaluates an uncertainty-aware machine learning surrogate for a solvent recovery superstructure. A deep ensemble of heteroscedastic neural networks predicts the feasibility, solvent recovery, purity and cost per kilogram of any given path through the superstructure. The predictive uncertainty is computed, decomposed and used to strategically acquire useful data as part of an active learning approach, thus simulating economical training on hand-crafted data in place of simulated data. The ensemble attains 99.9% feasibility classification accuracy and an RMS of ~ 0.02 on the recovery and purity fractions when trained on simulated flowsheet evaluations, with post-hoc variance scaling correcting a systematic underconfidence. I discuss the limitations of this approach, in particular its failure on trivial edge cases and its lack of interpretability, and argue that future work should focus on inherently interpretable architectures, such as generalised additive models, before deployment in high-stakes settings.

1 Introduction

1.1 Framing the problem

Yeomans and Grossmann (1999) originally proposed the superstructure approach to systems design, contrasting it to sequential design, which has been shown to produce suboptimal results. They frame the superstructure as a MINLP optimisation problem with a global optimum that can be found using computational methods. Each unit operation is thereby modelled in terms of mathematical functions taking a well-defined set of parameters as input, thus allowing the use of an optimisation framework. Chea et al. (2020) applied this approach to solvent separation with economic factors, achieving competitive results. The goal of this project is to investigate opportunities for applying machine learning to finding desirable paths through their proposed superstructure. The superstructure they proposed will be the one that I will consider throughout this project. It is shown in Figure 1.

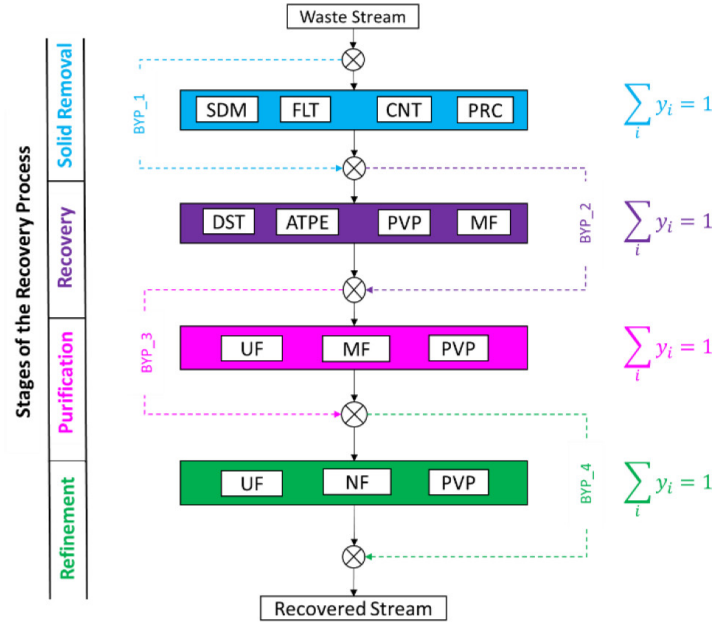


Figure 1: A visualisation of the superstructure that will be considered throughout this project. It was originally proposed by Chea et al. (2020). Reproduced from E. A. Aboagye et al. (2021).

1.2 Proposed solution

A machine learning surrogate offers two things an optimiser does not. Firstly, it provides a framework that can be expanded given new data, without having to explicitly model the underlying logic. This is the great power of machine learning, but it poses some issues that will be discussed later. Secondly a machine learning approach could estimate the uncertainty in a prediction. While the directly optimised superstructure promises solid results and is inexpensive to compute, it provides only a point estimate. This will often not suffice for a decision maker in a deployment setting. They will want to, in addition, receive an estimate of the consulting system’s confidence in the decision.

Thus, the goal of this project will be to use machine learning to do both these things. It should allow for expansion using hand-crafted data and provide an uncertainty estimate while doing so. Possible strategies for doing this will be discussed in the following sections.

1.3 Structure of the report

This report will first outline machine learning as a technique in Section 2.1, providing a brief overview of what it is, what it’s good at and where it struggles. Deep learning as a subset of machine learning is discussed in 2.2 and the issue of model interpretability, i.e., the transparency of a system’s predictions, is discussed in Section 2.3. This is followed by a brief literature review on the use of machine learning for process synthesis. Section 3 explains in more detail my proposed approach to using machine learning,

including information on the implementation, data generation and evaluation. Section 4 contains results, both qualitative and quantitative. These results are then analysed in Section 5 and a personal assessment of the project is provided. Section 6 is a summary of the work including an outlook and recommendations for future research.

1.4 Contributions

This project makes the following contributions:

- A **data generator** using an inversion of the framework proposed by Chea et al. (2020) to estimate the metrics for any two-solvent waste-stream passing through the separation superstructure.
- A **deep heteroscedastic ensemble** trained on the generated data, including theoretically sound uncertainty decomposition and calibration.
- An **active learning** implementation using the maximisation of the predicted epistemic uncertainty as an acquisition function.¹
- An **interactive GUI** for evaluating the model on any custom waste stream (under some constraints).
- A **roadmap for future research**.

2 Background

2.1 Machine Learning

Machine learning is the subset of artificial intelligence that concerns itself with using data to learn the parameters θ of a function $f_{\theta,\lambda}$ with hyperparameters λ . The idea is to build a statistical model that learns from existing data, the goal being to generalise to unseen data, performing tasks not explicitly programmed. There are three different machine learning paradigms:

- supervised learning, which learns input-output mappings
- unsupervised learning, which approximates the probability distribution of unlabeled data
- reinforcement learning, which trains an agent to navigate an environment of connected states of different value in a maximally profitable manner

I will only focus on supervised learning, as the other approaches are not relevant to the task at hand. Supervised learning can be further divided into

- classification, mapping n -dimensional input data $x_1, \dots, x_N \in \mathbb{R}^n$ to corresponding discrete output labels $y_1, \dots, y_N \in \mathbb{N}$
- regression, which maps n -dimensional input data to m -dimensional, real-valued outputs $y_1, \dots, y_N \in \mathbb{R}^m$

The approximated function outputs predictions $f_{\theta}(x_i) = \hat{y}_i$, whose quality can be measured by some loss function $\ell(\hat{y}_i, y_i)$ if the true output y_i is known.

¹This produces sub-par results, as later shown. Despite this, it is well-suited to act as a test-bed for more refined implementations.

2.2 Deep Learning

There are many different machine learning algorithms. Over the past few years, deep learning has become the most prevalent, due to its ability to outperform other algorithms on increasingly complex and high-dimensional domains, as described by Sejnowski (2020). In conjunction with the technological advances allowing for the feasible training of very deep networks with millions of parameters, this has allowed for an influx of research bordering on a technological revolution.

Deep learning is the subset of machine learning which uses multi-layered neural networks. A neural network f_θ has parameters

- $W_1, \dots, W_L$ are L weight matrices, one per layer.
- $b_1, \dots, b_L$ are bias vectors, one per layer.

The approximated function is then evaluated as

$$\begin{aligned} h_1 &= g_1(W_1x + b_1) \\ &\vdots \\ h_L &= g_L(W_L h_{L-1} + b_L) \\ \hat{y} &= g_L(W_L h_L + b_L) \end{aligned}$$

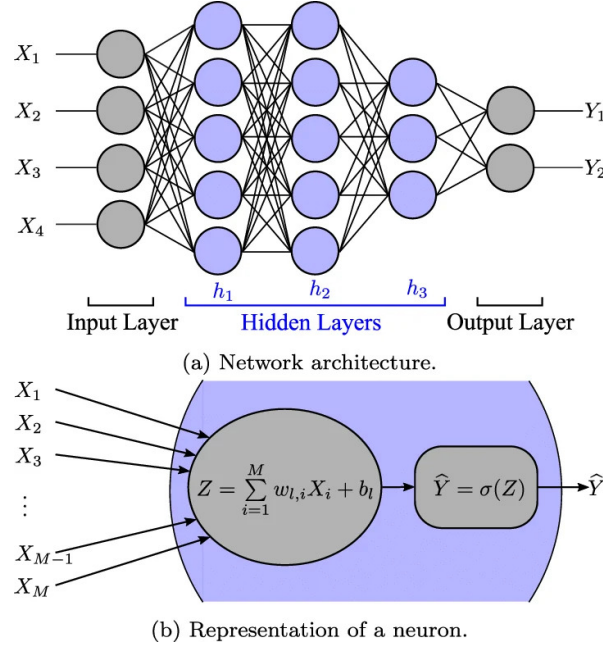


Figure 2: Visualisation of a neural network. Reproduced from Ihme et al. (2022).

The g or σ are non-linear activation functions. They allow for approximating non-linear functions, as without the network would decompose to a linear function. The most common activation function is $ReLU(x) = \max(0, x)$.

Graphics processing units (GPUs) are well suited for deep learning, as they're optimised for many concurrent linear operations, which is precisely what happens during the training process. The training involves computing the gradient of the loss function with respect to each of the parameters, thus finding the direction of steepest increase, then subtracting the gradient (scaled by a learning rate η) from the parameters in order to reduce the predicted loss in the next generation.

$$\theta_{t+1} \leftarrow \theta_t - \eta \nabla_{\theta} \ell(\hat{y}, y) \quad (1)$$

Equation 1 describes the iterative update step of the parameters for new step $t + 1$. In practice the gradient descent step is performed stochastically using batches of data with a given batch size. The stochastic gradient descent (SGD) formula is trivially derived from Equation 1.

2.2.1 Hyperparameters

Most learning algorithms require so-called hyperparameters λ to be set. The approximated function $f_{\theta, \lambda}$ directly depends on λ . Many hyperparameters pertain to regularisation, i.e., ensuring the approximated function generalises and does not overfit the training data. Some relevant hyperparameters are elaborated in this section.

The **learning rate** η from Equation 1 scales the gradient subtraction from the parameters. It should be set such that the update steps do not overshoot the target. Too small, and the algorithm gets stuck in local minima. Too large, and the algorithm bounces around without ever settling close to a minimum.

The **batch size** describes the number of simultaneous data points used in SGD. The larger it is, the smoother the gradient becomes, as averaging over more samples implies less noise. Larger batch sizes also come at an increased computational cost. Smaller batches are noisier, yet can act as regularisation and can allow for escaping local minima.

Dropout is a regularisation technique introduced by Srivastava et al. (2014). It involves setting connections to zero during training with some probability p , as sort of a handicap, then not skipping any connection during evaluation and scaling the output accordingly. The probability p can then be considered a hyperparameter.

Weight decay is another regularisation technique, one of the first to be commonly used for neural networks. It was introduced by Krogh and Hertz (1991) and suggests incorporating a weighted L2 norm of the weight vectors into the loss function, thus penalising large weights. This results in a network that is incentivised to prioritise small weights and thus simple solutions. It is a form of Occam’s razor that counteracts overfitting. The weight of the L2 norm acts as a hyperparameter, influencing the degree of influence that weight decay has on the loss function.

2.2.2 Surrogate Models

Henao and Maravelias (2011) propose using deep neural networks to replace intractable unit models. They provide examples of this approach working well and thereby lay the foundation for this approach. A key contribution lies in their analysis of the required input variables of each surrogate model. They introduce a formal procedure to identify a set of variables with minimal cardinality that makes the procedure solvable. E. Aboagye et al. (2025) use machine learning as a surrogate for expensive life cycle inventory analysis (LCIA) simulations, achieving competitive results. Granacher et al. (2021) construct and compare different machine learning surrogate models for expensive superstructure simulators in the domain of chemical engineering. They propose an active learning approach, achieving the best results using neural networks.

2.2.3 Active Learning

Active learning involves the model training on iteratively collected data, whereby the data is strategically collected in a way that is estimated to be most useful for training. The usefulness of data is quantified by some acquisition function $J(X | f_{\theta})$. Useful data tends to be the data for which the model is maximally uncertain. Active learning can be used to strategically feed data into the training process, in order to minimise the amount of data required. This is particularly useful if data generation is expensive. The following is an active learning algorithm as suggested by Gal et al. (2017):

1. Generate a training set X_0 and a model f_{θ_0} with initial parameters θ_0 .
2. Train f_{θ} on X_0 .
3. Repeat until some criterion is met:
 - (a) Select $X' = \arg \max_X J(X | f_{\theta_t})$ of a certain pre-determined cardinality as new data.
 - (b) Append this data to the initial training data to obtain $X_{t+1} \leftarrow X_t \cup X'$.

- (c) Initialise a new model with random weights and train on the combined data X_{t+1} to obtain $f_{\theta_{t+1}}$.
- (d) Update the iteration counter as $t \leftarrow t + 1$.

Active learning lends itself well to the problem at hand, as the architecture generates its own data and can do so within the loop, without the need for external intervention. For future research, it could be interesting to generate the data by hand, through physical experiments in a laboratory. This framework can easily be extended with a human-in-the-loop. An active learning approach would allow for a computational system that requests the data it still requires in order to become more confident.

2.2.4 Uncertainty Quantification

Deep neural networks tend to be severely overconfident (Guo et al., 2017). This is an issue for their use as part of the design of costly and high-stakes procedures, such as the one at hand, involving toxic chemicals and significant shareholder value. To mitigate, deep learning models in high-stakes applications should do two things:

1. quantify the uncertainty (or confidence) of their predictions
2. be well calibrated, i.e., make predictions with an appropriate degree of confidence

An obvious approach to uncertainty quantification is Bayesian. Instead of finding fixed model parameters θ , we can approximate a posterior distribution over the weights using Bayes' theorem.

$$P(\theta \mid x_1, \dots, x_n) = \frac{P(x_1, \dots, x_n \mid \theta)p(\theta)}{P(x_1, \dots, x_n)} \quad (2)$$

This is mathematically elegant, but, in practice, expensive to compute and it has been argued that Bayesian deep learning is prone to sub-optimal results (Wenzel et al., 2020).

Another, more straight-forward approach involves finding a probability distribution over the outputs. Uncertainty can be quantified by replacing a deterministic point estimate prediction $f_\theta(x) = \hat{y}$ by a predicted probability distribution (i.e., a likelihood) $p(x \mid \theta)$ over the space of possible outputs. The output can then be sampled as $\hat{y} \sim p(x)$. Its variance σ^2 quantifies the degree of uncertainty. This is called heteroscedastic if σ^2 is specific to each data point.

Nix and Weigend (1994) wrote a seminal paper arguing that a neural network $f_\theta(x)$ trying to approximate inherently noisy data will never be able to capture the noise $n(x)$ for new data $d(x)$. They frame the problem as

$$d(x) \approx f_\theta(x) + n(x) \quad (3)$$

implying that the true observed data consists of the approximation plus noise. With this understanding, the point estimate essentially becomes an estimate $\hat{\mu}$ of the true mean μ of a Gaussian distribution. The variance can then be computed in a per-datapoint (i.e., heteroscedastic) manner by maximising the Gaussian likelihood. In practice, neural network is fitted with a heteroscedastic head, outputting a pair $(\hat{\mu}_i, \log \sigma_i^2)$ per output y_i and is trained to minimise the negative log likelihood as a loss function. The per-datapoint approach is useful for determining a model's degree of confidence.

A non-Bayesian alternative building upon the previously discussed approach is suggested by Lakshminarayanan et al. (2016), who propose deep ensembles for uncertainty quantification. A deep ensemble is a collection of M deep neural networks with identical architecture, but different (randomly selected) initial parameters. Each model has a heteroscedastic head parametrising a Gaussian distribution, trained on some scoring function. The models are then trained on the same data, each learning a different set of parameters due to their differing initial parameters. The trained models are then treated as a uniformly weighted mixture model

$$p(y \mid x) = \frac{1}{M} \sum_m p_{\theta_m}(y \mid x) \quad (4)$$

The results of this approach can then directly be used to decompose the resulting uncertainty into aleatoric uncertainty (inherent to the data, thus irreducible) and epistemic uncertainty (due to an insufficiently informed model, thus reducible). This distinction is provided by the law of total variance:

$$Var(y | \theta) = E(Var(y | \theta)) + Var(E(y | \theta)) = \underbrace{Var(\mu)}_{\text{epistemic uncertainty}} + \underbrace{E(\sigma^2)}_{\text{aleatoric uncertainty}} \quad (5)$$

From this formula one can trivially derive that the sample variance of the mean

$$Var(\mu) \approx (M - 1)^{-1} \sum_{m=1}^M (\hat{\mu}_m - \bar{\mu})^2 \quad (6)$$

describes the predicted epistemic uncertainty and the sample mean of the variance

$$E(\sigma^2 I) \approx M^{-1} \sum_{m=1}^M \hat{\sigma}_m^2 \quad (7)$$

approximates the aleatoric uncertainty. The epistemic uncertainty now tells us exactly to which degree a given data point can still benefit the model. It is exactly the part of the variance that can still be influenced by further training. For this reason, the deep ensembles can elegantly be combined with active learning, the approximated epistemic uncertainty being used as the acquisition function $J(X | f_\theta)$.

Granacher et al. (2021) use an approach called Monte Carlo dropout for uncertainty quantification. I decided against using this approach, as both Lakshminarayanan et al. (2016) and Beluch et al. (2018) argue convincingly that it is outperformed by deep ensembles.

2.3 Model Interpretability

Deep learning, with all its undeniable power (under the right assumptions), comes with two significant drawbacks.

1. Its computational cost renders the research highly dependent on financial means and supply chains (Strubell et al., 2019).
2. A deep neural network functions essentially as a blackbox (Lipton, 2016).

The first point is important as part of an overview of the field. For this project, the model in question will be sufficiently inexpensive that it can be trained on a commercial GPU and used on an ordinary laptop. Thus, this point will not be further considered.

The second point is more important for this project. The complex linear functions with non-linear activation functions create computations that are so far abstracted from logical causation that very little transparency is provided in terms of how the model made its prediction. Post-hoc interpretation is possible to a certain degree, but requires caution. Rudin (2018) argues that explanations of naive neural networks, which aren't explicitly built to be interpretable, are futile to the point of potential danger, as they legitimise potentially misleading output.

An interpretable model makes predictions in a way that the process leading to the prediction can be understood by a human. The core idea is that it is not only important what a model predicts, but rather how it makes its predictions.

An example of an interpretable model is a decision tree. It takes the form of a branching structure where each internal node splits on a feature, and you follow a path down to a leaf. Different branches can test different features in different orders depending on earlier answers. The decision-making process can be traced question-by-question. An approach for finding optimal rule lists, a subset of decision trees, is provided by Angelino et al. (2018).

Unfortunately the decision tree is limited in its power. A more powerful, yet still interpretable approach, is provided by explainable boosting machines (EBMs), as proposed by Nori et al. (2019). They are essentially a fancy and modern version of the generalised additive model (GAM) (Lou et al., 2012). The idea involves modelling the output, either regression or classification, as a sum of univariate shape functions $f_1(x_1) + \dots + f_n(x_n)$, each operating on a single feature x_i .

$$g(y) = \beta + \sum_i f_i(x_i) \quad (8)$$

β is thereby an intercept and $g(y)$ is called the link function, which is the logistic function for classification and the identity function for regression. GAMs thus do not model high-order feature interactions. This makes them impractical for complex high-dimensional data, such as images, video and text. A more advanced version called GA²M was proposed by Lou et al. (2013), which incorporates few pairwise interactions to increase the model’s accuracy without sacrificing interpretability. The feature pairs are ranked using the proposed FAST algorithm, then incorporated into the model as

$$g(y) = \beta + \sum_i f_i(x_i) + \sum_{i \neq j} f_{ij}(x_i, x_j) \quad (9)$$

It has been argued that for low-dimensional tabular data the performance margin is insubstantial. In a very recent review paper, Doohan et al. (2026) conclude that deep learning offers no performance benefit over GAMs for tabular data. All the while the decision process learned by an intelligible model can be verified by experts as a sanity check. Any unintuitive rule can be screened and, if dangerous, removed. Caruana et al. (2015) mention the correlation between asthma and surviving pneumonia as an example. In practice, people suffering from asthma tend to have a higher chance of surviving pneumonia. This unintuitive results stems from the fact that they tend to receive more intense treatment than the average person. A deep learning model would have simply learned this as a fact. An interpretable model, on the other hand, isolates this learned correlation, whereafter it can be screened and removed.

The INTERPRETML library offers a fast and convenient EBM implementation (Nori et al., 2019).

3 Methodology

I constructed a heteroscedastic, thus uncertainty-aware deep ensemble surrogate model using the framework proposed by Chea et al. (2020) as an oracle for data generation. This oracle acts as a stand-in for the laboratory experiments that would supply the data in a production setting. The following sections will describe and justify this approach.

3.1 Data

3.1.1 Generation

I developed a Python program that is used for the data generation. It builds upon the formulas specified in the supplement of Chea et al. (2020), differing from their implementation as it allows for computing an evaluation of any two-solvent waste stream. Thus, it can be understood as the framework inverted.

Their implementation takes a waste stream as input, then frames it as a MINLP optimisation problem and outputs a solution as a path through the superstructure. My implementation, in contrast, takes both a waste stream and a path through the superstructure as input, outputting a probability distribution over predicted metrics on cost, purity and fraction of the recovered solvent and feasibility of the suggested path given the waste stream composition. This allows for generating a vast dataset of waste streams, various paths through the superstructure and resulting metrics, as required by a supervised learning model.

3.1.2 Selection

The waste stream is assumed to be comprised of two solvents (one of which is the target), salt, water and solids at a specified temperature. Firstly, the waste stream is characterised by the following parameters for each component:

- flow rate in kg/h
- the relative volatilities α_1 and α_2 between the target solvent and
 1. the second solvent
 2. water
- density in kg/m³
- molecular weight in g/mol

- boiling point in Kelvin
- heat of vaporisation in kJ/kg
- liquid heat capacity in kJ/kg*K
- saturation pressure in Pa
- $\log_{10}$ octanol-water partition coefficient

These parameters were extracted from the formulas in the supplement. Secondly, the input contains a path through the superstructure. For each step, every unit operation is assigned an index. The index 0 always corresponds to a bypass. Thirdly, the temperature is provided in °C.

Index	Solids removal	Recovery	Purification	Refinement
0	bypass	bypass	bypass	bypass
1	sedimentation	distillation	distillation	distillation
2	centrifugation	pervaporation	pervaporation	pervaporation
3	filtration	aq. two-phase extraction	ultrafiltration	ultrafiltration
4	—	—	—	microfiltration

Table 1: Index-based encoding of the superstructure path. Index 0 always denotes a bypass of that stage. The resulting path space contains $4 \times 4 \times 4 \times 5 = 320$ distinct pathways.

The output consists of

1. a feasibility prediction
2. a predicted target solvent recovery fraction
3. a predicted purity of the recovered solvent as fraction
4. an estimated cost per kg of recovered solvent

3.1.3 Preprocessing

The path through the superstructure is represented as one-hot encoded vectors. Each step consists of a vector of zeros, with a one at the index representing the unit operation from Table ?? . The bypass is thereby not modelled as an index, i.e., if a step is bypassed, the vector consists only of zeros.

The non-integer data is preprocessed by applying z -score standardisation to each feature. This standardises each feature x_i as

$$x'_i = \frac{x_i - \bar{\mu}_i}{s_i} \quad (10)$$

for sample mean $\bar{\mu}$ and sample standard deviation s . As such, the data is rescaled to the assumption of $x' \sim \mathcal{N}(0, 1)$. This leads to more stable training and faster convergence, as argued by Al-Faiz et al. (2019).

3.2 Model Architecture

The machine learning model should fulfil the following criteria:

1. Continuous flowsheet evaluation given an input waste stream for changing waste stream parameters. The outputted evaluation should include all the outputs from above.
2. A well-calibrated uncertainty estimate.
3. The model should be sufficiently computationally inexpensive to allow for real-time parameter augmentation, e.g. for visualisation purposes.

As suggested in Section 2.2.4, deep ensembles as described by Lakshminarayanan et al. (2016) provide a solid framework for uncertainty estimate. The disadvantage of deep ensembles is that the number of model parameters must be scaled by a factor of M . This should not pose an issue for our purposes, as our model has relatively small input and output dimensions of $x \in \mathbb{R}^{24}$ and $y \in \mathbb{R}^4$. Thus, our model will not be sufficiently large for the increased number of parameters to become an issue.

The goal is to develop a model that can predict different metrics. This allows for more efficient training and simpler deployment. Each predicted output gets its own head and loss function. As probability distributions are predicted, the heads contain their parameters. The binary feasibility classification corresponds to a Bernoulli distribution with a single parameter p , which is approximated as a logit, such that the probability is obtained by passing the output through a sigmoid function as $\sigma(\hat{y})$. The three regression tasks each output $\hat{\mu}$ and $\log \hat{\sigma}^2$ approximating a Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$ each. Predicting the logarithm of the variance is standard practice for heteroscedastic neural networks, for reasons ranging from positivity constraints to numerical stability. The log variance outputs have been clamped to the range $[-10, 10]$, to avoid exploding gradients and numerical overflow.

Due to the relatively low dimensionality and small state space one can assume that the number of layers and their size will not significantly alter the performance. Thus, I settled on a simple architecture consisting of four shared layers and three layers per head, which manages to fit the data with no issues.

3.3 Active Learning

The active learning implementation follows the algorithm by Gal et al. (2017) outlined in Section ?? . The acquisition function works by sampling a large amount of data, passing it through the current generation’s ensemble and computing the epistemic uncertainty for every data point separately. The sampled data is then ranked according to the epistemic uncertainty, whereafter the top $\lfloor k \cdot (1 - \epsilon) \rfloor$ are selected and mixed with $\lfloor k \cdot \epsilon \rfloor$ randomly generated data points. $\epsilon = 0.5$ is an exploration parameter borrowed from the ϵ -greedy policy from the reinforcement learning literature (Wunder et al., 2010). It should counteract the issue of sampling bias, which occurs when an increasingly large fraction of the training data is not sampled i.i.d., thus breaking a fundamental assumption in machine learning (Dasgupta & Hsu, 2008). Simply selecting the top k uncertain data points in each generation thus leads to unstable results. More complex solutions have been proposed (e.g. by Beygelzimer et al. (2009)) and could be relevant for future work.

3.4 Metrics

As the goal involves forecasters and thus predicting distributions, one must train on scoring rules. A scoring rule $S(p_\theta, (y, x))$ measures the quality of predictive uncertainty by assigning a numerical score to a predictive probability distribution, punishing predictions of low accuracy (with respect to the given data) with high scores. A scoring rule is proper if its minimisation implies that the evaluated forecaster is equal to the ideal forecaster (Gneiting et al., 2007). Differentiable proper scoring rules have been shown to perform well as loss functions as part of training a neural network (Lakshminarayanan et al., 2016).

This project uses Gaussian negative log likelihood as a scoring rule for regression and binary cross-entropy loss for classification. In addition, the Brier score and root mean-squared error are used as validation metrics, describing the model’s point estimate performance. The infeasible regression targets are masked out, their values are set to zero and carry no meaning.

3.5 Calibration

The calibration plots show the statistical consistency between the predictions and the observations (Gneiting et al., 2007). In a well-calibrated classifier, data points that are classified within a particular probability range will have a similar probability of being correctly classified. This is important, as we don’t want the model to be overconfident in its predictions. We want it to be able to accurately gauge its uncertainty.

An extension of this idea to regression was proposed by Kuleshov et al. (2018). They do so by taking a list of assumed nominal probabilities $p_1, \dots, p_n$ and computing the corresponding confidence probabilities as

$$\hat{p} = \frac{\sum_t \mathbb{I}\{y_t \leq F_t^{-1}(p)\}}{T} \quad (11)$$

The quantile function $F_t^{-1}(p)$, i.e., the inverse cumulative distribution function of the predicted distribution f_θ , is computed per-sample as

$$F_t^{-1}(p) = \mu_t + z_p \cdot \sigma_t \quad (12)$$

for standard normal p -quantile z_p and a Gaussian f_θ . This results in a plot of the cumulative distribution function of the probability integral transform (PIT), which, if the model is well-calibrated, should correspond to a uniform distribution in the range $[0, 1]$. Thus, the ideal model would result in a perfectly diagonal calibration plot (Gneiting et al., 2007).

In essence, Kuleshov et al. (2018) argue that a model is well-calibrated if targets fall below the p -quantile with a probability of p .

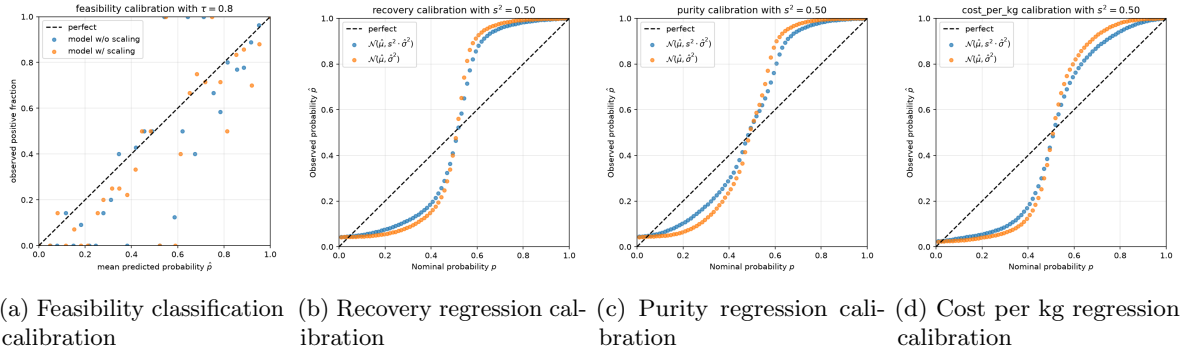


Figure 3: Calibration plots as defined by Kuleshov et al. (2018) for all four predicted metrics. The x describes a range of nominal probabilities and the y -axis describes the observed probabilities per nominal probability bin.

One can observe in Figure 3 that the models tend to be systematically and globally underconfident without scaling, i.e., they predict excessive variance, thus are not well calibrated. As the underconfidence is global and centered around $p = 0.5$, this can easily be fixed by scaling the predicted variance.

The feasibility classification calibration can be adjusted by temperature scaling (Guo et al., 2017). This involves scaling the outputted logits $\log p$ with a temperature τ .

Extending this approach to regression, Levi et al. (2019) propose a technique called STD scaling. It involves, per head, computing the scalar

$$s^2 = \frac{1}{T} \sum_t \frac{(y_t - \hat{\mu}_t)^2}{\hat{\sigma}_t^2} \quad (13)$$

using a separate calibration set, then scaling the variance of the predicted distribution as $\mathcal{N}(\hat{\mu}, s^2 \cdot \hat{\sigma}^2)$.

The scaling values s^2 and τ are selected based on their performance on a separate fresh calibration set, are overlaid in Figure 3. One can see that scaling significantly improves the miscalibration. Future research should attempt to fit more informed scaling values by using the proposed approaches. In the scope of this project there is simply too little time to do so.

3.6 Hyperparameter Tuning

I tuned weight decay, dropout probability (equal at every layer), batch size and learning rate using a random sampling approach, as suggested by Bergstra and Bengio (2012). The results suggest that the batch size shows a strong correlation with the loss function. The lowest loss occurs with a batch size of around 32. Batch sizes of larger than 512 lead to exploding losses.

One can observe a similar tendency in the learning rates. High learning rates (0.1 and larger) lead to high losses. The sweet spot lies around $\eta = 3 \times 10^{-4}$ for a batch size of 32. As expected, the larger batch sizes require larger learning rates in order to perform well.

Weight decay seemed to not play a significant role in the validation performance. A value of 0.1 seemed to perform well at times, but so did 0. Dropout performed terribly on the validation data. Setting $p = 0$ led to the best results by an impressive margin.

Thus the optimal configuration, i.e., the set of hyperparameters that performed best in the experiment, involves a batch size of 32, a learning rate of $\eta = 3 \times 10^{-4}$, a weight decay with scalar 0.01 and no dropout.

3.7 Ensemble Training

The initial ensemble was trained with the hyperparameters specified in Section 3.6 with $M = 5$. A five epoch learning rate warmup was used with cosine annealing thereafter, in conjunction with the AdamW optimiser, proposed by Loshchilov and Hutter (2019). The training, validation and test datasets consist of 10^7 data points generated using Latin hypercube sampling. The metrics in Figure 4 show that the model not only fits the data, but also generalises well, without any significant discrepancy between the training and validation losses.

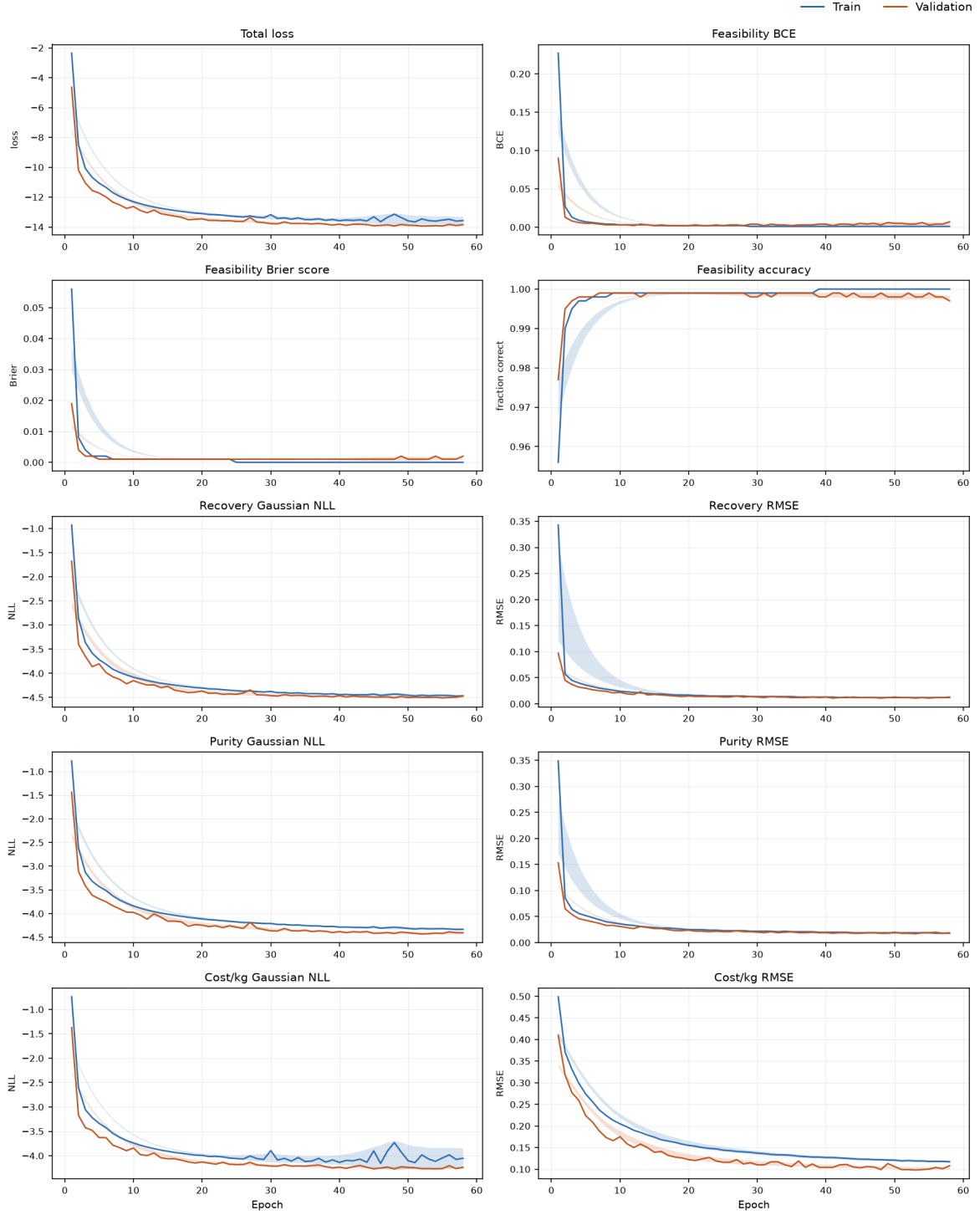


Figure 4: Training and validation metrics over training epochs for the initial ensemble. The bands around the line correspond to the sample standard deviation s of the ensemble predictions. The total loss refers to the sum of the score functions (i.e., Gaussian negative log likelihood (NLL) and binary cross-entropy (BCE)). The point-estimate metrics, such as the root mean-squared error (RMSE) and the prediction accuracy are used only for evaluation.

The results suggest that from epoch 80 onwards, the standard deviations of the NLL and the cost/kg metrics start to grow. This suggests that early stopping around that point suffices for future training (and active learning).

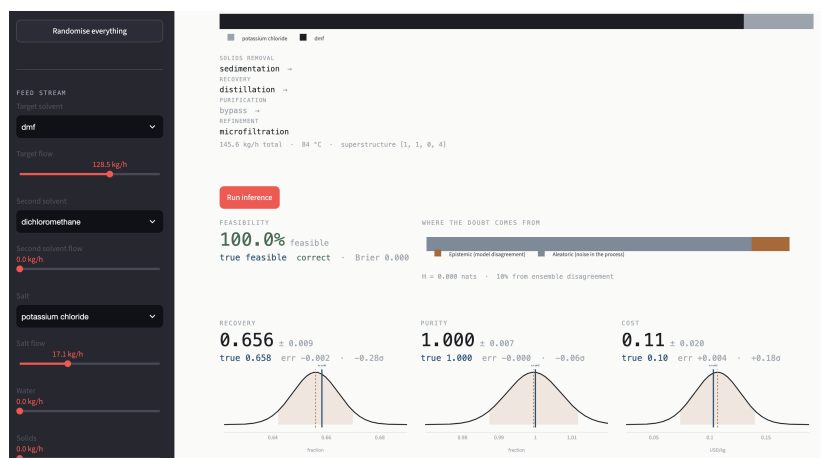


Figure 5: A screenshot showing the visualiser.

3.8 Visualiser

I implemented a visualiser that runs in the browser. It allows for constructing a waste stream under the assumptions of the model. The application then runs the stream through both the model and the framework adapted from Chea et al. (2020), comparing the results.

4 Results

4.1 Active Learning

I ran active learning with a small initial dataset consisting of 10 000 elements for different ϵ values. Each generation acquired an additional data set equivalent in size to 10% of the initial training set. The results can be observed in Figure 6. Surprisingly, $\epsilon = 1$ yielded results that are on-par with $\epsilon = 0.5$. As the prior entails pure exploration, the results implies that the acquisition using a theoretically sound uncertainty decomposition does not improve performance over random acquisition. This is a surprising result.

Per generation, the model trained for 60 epochs or until conversion (represented by 10 epochs without an increase in total validation loss). Per generation, a total of 100 000 new data corresponding to 10% of the initial dataset, selected from a data pool of 10^6 randomly generated data points based on their epistemic uncertainty. The results can be seen in Figure 6.

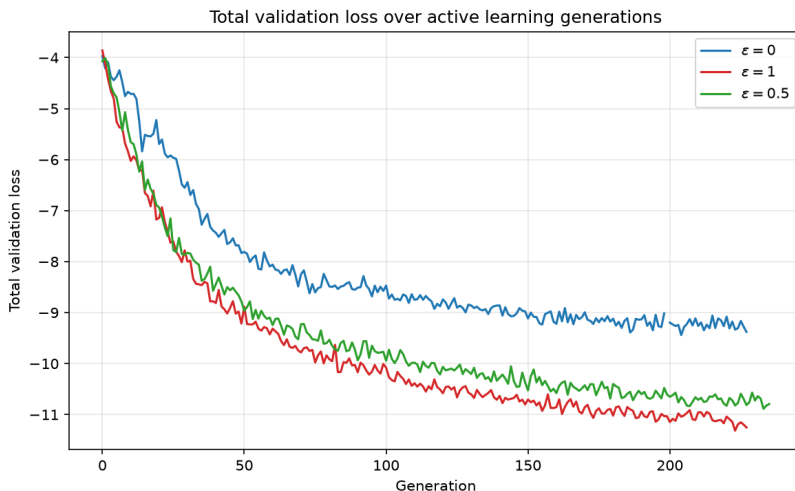


Figure 6: The total loss on the validation data plotted over the number of active learning generations for different ϵ values. Less is thus better. Epoch 0 refers to the validation loss of the initial model.

One can see that active learning leads to a marginal improvement in total validation loss, yet not consistently. The sampling bias, while empirically improved with ϵ -greedy, is still prevalent and leads to increased loss for later generations. Better strategies might be investigated in future work.

4.2 Test Losses

Table 2 shows the loss values of the ensemble after active learning on the withheld test data. One can clearly see, when comparing the values to Figure 4, that the ensemble generalises well to the unseen data. The RMSE for the cost per kg prediction is comparatively higher as the range of its regression values is larger than the fractions in $[0, 1]$. Feasibility is correctly predicted 99.9% of the time with hardly any deviation between the models, thus high confidence.

Target	Metric	Value
Feasibility	BCE	0.002 ± 0.0
	Brier	0.000 ± 0.0
	Accuracy $\in [0, 1]$	0.999 ± 0.0
Recovery	NLL	-4.538 ± 0.007
	RMSE	0.012 ± 0.0
Purity	NLL	-4.441 ± 0.014
	RMSE	0.018 ± 0.0
Cost per kg	NLL	-4.282 ± 0.009
	RMSE	0.183 ± 0.003

Table 2: Ensemble performance on the test data. Values are mean $\pm$ standard deviation across ensemble members. The zero-standard deviations are in reality <0.001 .

The negative NLL values are to be expected, as the negative log likelihood of a continuous density can be negative.

4.3 Qualitative Analysis

The results fit the data well when observing the problem from a quantitative lens. A caveat that is only observable during manual evaluation using the visualiser is that the ensemble struggles with obvious edge cases. Setting all layers of the superstructure to bypass and the stream to pure target solvent leads to consistently under-estimated purity values from an otherwise impressively accurate model. For

a human, even without subject expertise, the conclusion seems obvious that the purity of pure and untouched solvent should be 100%. This mirrors the conclusion drawn by Marcus (2018), who argues that deep learning by its very nature struggles to represent causal relationships the way humans do. This must be taken into consideration for any deep learning application in a high-stakes environment. A neural networks learns statistical relationships, not causal logic.

5 Discussion

5.1 Heteroscedastic deep learning accurately predicts solvent separation metrics

Developing an uncertainty-aware model for analysing the outcome of different solvent separation flowsheets has to my knowledge not previously been attempted. Deploying an approach constructed in a similar way could allow for two main contributions. Firstly, it would allow for real-time visualisations of the trade-off between different paths. This could be further developed into a tool for use by decision makers in the chemicals and pharmaceutical industries. Secondly, it would provide a proof-of-concept baseline for an efficient approach to human-in-the-loop training. The model can request the data it requires by telling where it is still uncertain. The data could then be provided by experimental results in a laboratory.

5.2 Drawbacks

The main drawback of this approach is that in its current form it will only ever be as good as the model by Chea et al. (2020) providing the data. Also, as they mention, their framework is subject to a relatively high standard deviation of 30%. Thus, absolute costs should be treated with caution. This could be solved by providing manually crafted experimental data.

An obvious limitation of the current implementation is the strong assumptions underlining the stream composition. Future work could pertain to devising an architecture with more flexible inputs, allowing for the analysis of streams containing more than two solvents.

5.3 The case for interpretable models

This point cannot be stated without considering the larger implications of model interpretability. The model predicts with high accuracy, a factor it itself is capable of gauging. The problem is that we have no idea how it makes its prediction.

The ensemble method quantifies uncertainty, which is of use to decision-makers, yet little interpretability is provided inherently. When assigning e.g. a low cost to a given configuration, it can not tell you why it thinks that to be the case. This stands in contrast to the argument made by Rudin (2018), who argues that only interpretable models should be used for high-stakes decisions (which probably includes any industrial-scale decision involving chemicals), and that attempting to explain non-interpretable models is futile and potentially harmful, as post-hoc justification can create plausible, yet false explanations. The question regarding the benefit of a deep learning approach then emerges. The MINLP implementation by Chea et al. (2020) provides a level of interpretability that is hard to beat by any machine learning system. Before pursuing the recommendations put forward in this report, it must be considered what exactly the benefits of machine learning are worth the trade-off in interpretability. This might be justified by the increased efficiency of machine learning, as e.g. with Henao and Maravelias (2011). Yet in our case, that is no factor due to the MINLP in question computing within a fraction of a second.

As shown in Section 2.3, the GAM is not systematically outperformed by the neural network on low-dimensional tabular data like the one involved (Doohan et al., 2026). I thus strongly suggest attempting a similar outcome using a GAM or EBM, developed in a team involving both machine learning and chemical engineers. As then described by Caruana et al. (2015), the experts could analyse the learned decision rules and use their domain-specific knowledge to help the mathematician refine the model architecture. This could be done using InterpretML, a Python library acting as a unified framework for interpretable AI approaches (Nori et al., 2021).

Other possible interpretable machine learning approaches could involve rule lists (Angelino et al., 2018) or, more broadly, decision trees. E.g., a node in the tree could learn ask if the relative volatility between any of the compounds exceeds a certain approximated value that is learned to be the threshold for distillation to be feasible within certain constraints.

Finally, it would also be possible to, instead of replacing the entire superstructure evaluation with big neural network, replace the evaluation of individual unit operations by multiple, smaller networks. This is the approach suggested in Henao and Maravelias (2011). The networks themselves could thus be intransparent, yet modular and more easily interpretable by a domain expert.

In summary, future work seems promising, yet whatever approach is further investigated, domain-specific knowledge on chemical engineering must be part of the development process for architecture design and for the evaluation of data, predictions and possible errors.

6 Conclusion

6.1 Summary

I devised, implemented and evaluated a deep learning approach for evaluating solvent separation flowsheets given a superstructure proposed by Chea et al. (2020). The model predicts feasibility, recovery, purity and cost per kg for any path through the superstructure. It is uncertainty-aware, decomposing its predictive uncertainty into an aleatoric and an epistemic component. After post-hoc variance scaling, the uncertainty estimates are well calibrated. The model fits the data well, reaching a feasibility classification accuracy of 99.8% on unseen test data.

The epistemic uncertainty was used as the acquisition function for active learning. The improvement over random sampling was marginal and inconsistent, which I attribute to sampling bias. Still, the framework demonstrates a model that can request the data it needs. This becomes valuable once the simulated oracle is replaced by laboratory experiments, where each data point carries a real cost.

Two limitations remain: firstly, the model inherits the reported 30% cost deviation. Secondly, it lacks interpretability, predicting accurately, yet failing on trivial edge-cases, such as pure solvent passing through a series of bypasses, without being able to explain its prediction. This is a major drawback for high-stakes decision making.

6.2 Future work

Future research should thus pertain to the following subjects:

- Investigate the performance of more interpretable models, such as GAMs, using this implementation as a baseline and involving experts to evaluate the learned decision rules.
- Find more profitable acquisition strategies in the proposed active learning implementation.
- Use such strategies to build an interface in order to train on hand-crafted data.
- Perform proper calibration, using temperature and STD scaling to fit an informed τ and s^2 respectively.
- Extend the architecture to allow for arbitrary waste streams containing more than two solvents.

A Computational Specifications

All specified programs were executed on consumer-grade hardware with 32 GB RAM and an Intel i7 processor. A standard, consumer-grade CPU suffices for evaluating the model. The neural networks were trained on an Nvidia RTX 4080 GPU using the PyTorch framework (Paszke et al., 2019). All plots were generated using the `matplotlib` library. The code, written in Python 3.14.6, is publicly accessible under <https://github.com/maxjappert/ml-superstructure-solvent-recovery>.

B Report to Code Mapping

The data described in Section 3.1 is generated within the `solvent_recovery_data_generator/` folder. The results from Figure 3 and Section 4.2 can be reproduced by running `evaluate.py`. Its `main()` function has been prepared to execute all the relevant functions. The visualiser can be executed using `streamlit run visualiser.py`. Figure 6 can be reproduced by plotting the `.json` files saved by the `active_learner.py` containing the best validation loss per generation. Figure 4 was generated by manually transferring the printed outputs from `train.py` to Python lists.

References

- Aboagye, E., Longo, J., Conway, M., Pazik, J., Barkow, M., McMahon, M., Weil, B., Appiah, H. D., Hesketh, R., & Yenkie, K. M. (2025). Leveraging machine learning algorithms to predict life cycle inventory assessments (LCIA) to facilitate sustainable process design. *Computers & Chemical Engineering*, 201, 109217. <https://doi.org/10.1016/j.compchemeng.2025.109217>
- Aboagye, E. A., Chea, J. D., & Yenkie, K. M. (2021). Systems level roadmap for solvent recovery and reuse in industries. *iScience*, 24(10), 103114. <https://doi.org/10.1016/j.isci.2021.103114>
- Al-Faiz, M. Z., Ibrahim, A. A., & Hadi, S. M. (2019). The effect of z-score standardization (normalization) on binary input due the speed of learning in back-propagation neural network. *Iraqi Journal of Information & Communications Technology*, 1(3), 42–48. <https://doi.org/10.31987/ijict.1.3.41>
- Angelino, E., Larus-Stone, N., Alabi, D., Seltzer, M., & Rudin, C. (2018). Learning certifiably optimal rule lists for categorical data. *Journal of Machine Learning Research*, 18.
- Beluch, W. H., Genewein, T., Nurnberger, A., & Kohler, J. M. (2018). The Power of Ensembles for Active Learning in Image Classification. *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 9368–9377. <https://doi.org/10.1109/CVPR.2018.00976>
- Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13, 281–305. <https://www.jmlr.org/papers/volume13/bergstra12a/bergstra12a.pdf>
- Beygelzimer, A., Dasgupta, S., & Langford, J. (2009). Importance weighted active learning. *Proceedings of the 26th Annual International Conference on Machine Learning*, 49–56. <https://doi.org/10.1145/1553374.1553381>
- Caruana, R., Lou, Y., Gehrke, J., Koch, P., Sturm, M., & Elhadad, N. (2015). Intelligible models for HealthCare: Predicting pneumonia risk and hospital 30-day readmission. *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1721–1730. <https://doi.org/10.1145/2783258.2788613>
- Chea, J. D., Lehr, A. L., Stengel, J. P., Savelski, M. J., Slater, C. S., & Yenkie, K. M. (2020). Evaluation of Solvent Recovery Options for Economic Feasibility through a Superstructure-Based Optimization Framework. *Industrial & Engineering Chemistry Research*, 59(13), 5931–5944. <https://doi.org/10.1021/acs.iecr.9b06725>
- Dasgupta, S., & Hsu, D. (2008). Hierarchical sampling for active learning. *Proceedings of the 25th international conference on Machine learning - ICML '08*, 208–215. <https://doi.org/10.1145/1390156.1390183>
- Doohan, J., Kook, L., & Burke, K. (2026). Comparison of generalised additive models and neural networks in applications: A systematic review. *Expert Systems with Applications*, 309, 131082. <https://doi.org/10.1016/j.eswa.2025.131082>
- Gal, Y., Islam, R., & Ghahramani, Z. (2017). *Deep Bayesian Active Learning with Image Data* (1). <https://doi.org/10.48550/ARXIV.1703.02910>
- Gneiting, T., Balabdaoui, F., & Raftery, A. E. (2007). Probabilistic forecasts, calibration and sharpness. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 69(2), 243–268. <https://doi.org/10.1111/j.1467-9868.2007.00587.x>
- Granacher, J., Kantor, I. D., & Maréchal, F. (2021). Increasing Superstructure Optimization Capacity Through Self-Learning Surrogate Models. *Frontiers in Chemical Engineering*, 3, 778876. <https://doi.org/10.3389/fceng.2021.778876>
- Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). *On Calibration of Modern Neural Networks* (2). <https://doi.org/10.48550/ARXIV.1706.04599>

- Henao, C. A., & Maravelias, C. T. (2011). Surrogate-based superstructure optimization framework. *AIChE Journal*, 57(5), 1216–1232. <https://doi.org/10.1002/aic.12341>
- Ihme, M., Chung, W. T., & Mishra, A. A. (2022). Combustion machine learning: Principles, progress and prospects. *Progress in Energy and Combustion Science*, 91, 101010. <https://doi.org/10.1016/j.pecs.2022.101010>
- Krogh, A., & Hertz, J. A. (1991). A simple weight decay can improve generalization. *Advances in Neural Information Processing Systems*, 4. A Simple Weight Decay Can Improve Generalization
- Kuleshov, V., Fenner, N., & Ermon, S. (2018, July). Accurate Uncertainties for Deep Learning Using Calibrated Regression [arXiv:1807.00263 [cs.LG]]. <https://doi.org/10.48550/arXiv.1807.00263>
- Lakshminarayanan, B., Pritzel, A., & Blundell, C. (2016). *Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles* (3). <https://doi.org/10.48550/ARXIV.1612.01474>
- Levi, D., Gispan, L., Giladi, N., & Fetaya, E. (2019). Evaluating and calibrating uncertainty prediction in regression tasks [Version Number: 3]. <https://doi.org/10.48550/ARXIV.1905.11659>
- Lipton, Z. C. (2016). *The Mythos of Model Interpretability* (3). <https://doi.org/10.48550/ARXIV.1606.03490>
- Loshchilov, I., & Hutter, F. (2019, January 4). Decoupled weight decay regularization. <https://doi.org/10.48550/arXiv.1711.05101>
- Lou, Y., Caruana, R., & Gehrke, J. (2012). Intelligible models for classification and regression. *Proceedings of the 18th ACM SIGKDD international conference on Knowledge discovery and data mining*, 150–158. <https://doi.org/10.1145/2339530.2339556>
- Lou, Y., Caruana, R., Gehrke, J., & Hooker, G. (2013). Accurate intelligible models with pairwise interactions. *Proceedings of the 19th ACM SIGKDD international conference on Knowledge discovery and data mining*, 623–631. <https://doi.org/10.1145/2487575.2487579>
- Marcus, G. (2018). Deep learning: A critical appraisal [Version Number: 1]. <https://doi.org/10.48550/ARXIV.1801.00631>
- Nix, D., & Weigend, A. (1994). Estimating the mean and variance of the target probability distribution. *Proceedings of 1994 IEEE International Conference on Neural Networks (ICNN'94)*, 55–60 vol.1. <https://doi.org/10.1109/ICNN.1994.374138>
- Nori, H., Caruana, R., Bu, Z., Shen, J. H., & Kulkarni, J. (2021). Accuracy, Interpretability, and Differential Privacy via Explainable Boosting. *Proceedings of the 38th International Conference on Machine Learning*. <https://proceedings.mlr.press/v139/nori21a.html>
- Nori, H., Jenkins, S., Koch, P., & Caruana, R. (2019). InterpretML: A unified framework for machine learning interpretability [Version Number: 1]. <https://doi.org/10.48550/ARXIV.1909.09223>
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., . . . Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library [Version Number: 1]. <https://doi.org/10.48550/ARXIV.1912.01703>
- Rudin, C. (2018). Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead. <https://doi.org/10.48550/ARXIV.1811.10154>
- Sejnowski, T. J. (2020). The Unreasonable Effectiveness of Deep Learning in Artificial Intelligence. <https://doi.org/10.48550/ARXIV.2002.04806>
- Srivastava, N., Hinton, G., & Sutskever, I. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15, 1929–1958. <http://jmlr.org/papers/v15/srivastava14a.html>
- Strubell, E., Ganesh, A., & McCallum, A. (2019). *Energy and Policy Considerations for Deep Learning in NLP* (1). <https://doi.org/10.48550/ARXIV.1906.02243>
- Wenzel, F., Roth, K., Veeling, B. S., Światkowski, J., Tran, L., Mandt, S., Snoek, J., Salimans, T., Jenatton, R., & Nowozin, S. (2020). *How Good is the Bayes Posterior in Deep Neural Networks Really?* (2). <https://doi.org/10.48550/ARXIV.2002.02405>
- Wunder, M., Littman, M. L., & Babes, M. (2010). Classes of multiagent q-learning dynamics with epsilon-greedy exploration. *Proceedings of the 27th International Conference on Machine Learning (ICML-10)*, 1167–1174.
- Yeomans, H., & Grossmann, I. E. (1999). A systematic modeling framework of superstructure optimization in process synthesis. *Computers & Chemical Engineering*, 23(6), 709–731. [https://doi.org/10.1016/S0098-1354\(99\)00003-4](https://doi.org/10.1016/S0098-1354(99)00003-4)